PYTHON PROGRAMMING – ASSIGNMENT 1

Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences, Chennai

Student Name	Nithin Chakravarthi PS
Register Number	1912260031
Course Code	CSA0805 SLOT-A
Submission Date	23.10.2026
GitHub Repository Link	https://github.com/chakravarthi2008/CSA0805A_1912260031_NITHIN-CHAKRAVARTHI-PS
Faculty Name	Hari Priya Anandakumar

Individual Assignment | CO1 | Total Marks: 50

Contents

- Task 1 – Understand the Problem and Identify the Data
- Task 1 – Alternative Design and Complexity
- Task 2 – Values and Types
- Task 2 – Alternative Money Representations
- Task 3 – Variables and Constants
- Task 4 – Correct Expressions
- Task 5 – Read Input and Convert Types
- Task 6 – Organise Logic into Functions
- Task 7 – Build a Module
- Task 8 – Algorithmic Problem Solving
- Task 9 – Implement and Test the Calculator
- Task 10 – Compare Expectations with Execution
- Task 11 – Recommend and Defend the Final Design
- Task 12 – Re-evaluate and Adapt the Design
- Submission Checklist

Task 1 – Understand the Problem and Identify the Data

The Ksheera Village Milk Producers' Co-operative records milk deliveries twice a day. The calculator must determine the rate per litre from fat and SNF readings, calculate each delivery payment, and produce a correct ten-day total.

Category	Details and justification
Inputs	Farmer ID (string), litres (decimal), fat % (decimal), SNF % (decimal), shift (string), and delivery date/period. IDs remain text to preserve leading zeroes.
Outputs	Rate per litre, payment for each delivery, and the farmer's ten-day total.
Named constants	BASE_RATE = Rs 30.00; REFERENCE_FAT = 3.5; REFERENCE_SNF = 8.5; FAT_STEP_VALUE = Rs 0.40; SNF_STEP_VALUE = Rs 0.30; STEP_SIZE = 0.1; MIN_FAT = 2.0; MAX_FAT = 7.0.
Data types	Strings for identifiers and shifts; Decimal for percentage readings and money; integers for counts; Boolean values for validation results.
Rules	Quantity must be positive; fat must be from 2.0% through 7.0%; values must be numeric; shift must be M or E; invalid entries are rejected; each payment is rounded to the nearest paisa.
Assumptions	Readings represent percentages; each delivery is rounded to two decimal places before adding to the ten-day total. The assignment does not specify an SNF range, so a provisional range must be confirmed with the co-operative.

Alternative Design and Complexity

Creating separate variables such as litres1, fat1 and snf1 for every farmer produces repetitive code and is difficult to maintain. A list of records such as (farmer_id, litres, fat, snf, shift) can be processed in a loop. Processing n records takes O(n) time. Storing all records takes O(n) space; maintaining only a running total requires O(1) additional space. A record-based design scales more cleanly from 300 farmers to 150,000.

Task 2 – Values and Types

```
>>> type(10)       # <class 'int'>
>>> type(10.0)     # <class 'float'>
>>> type("10")     # <class 'str'>
>>> type(True)     # <class 'bool'>
>>> type([10, 8.5]) # <class 'list'>
>>> 7 / 2          # 3.5
>>> 7 // 2         # 3
>>> 7 % 2          # 1
>>> -7 // 2        # -4
>>> 0.1 + 0.2 == 0.3 # False
```

The division operator / returns a fractional result, // performs floor division, and % returns the remainder. Floor division rounds down toward negative infinity, which explains why -7 // 2 is -4. Binary floating-point cannot represent many decimal fractions exactly, so 0.1 + 0.2 == 0.3 evaluates to False.

For money, integer paise or Decimal created directly from strings is preferable. Decimal preserves the decimal input exactly; converting a float to Decimal imports the float's approximation.

Completed Decimal Total

```
from decimal import Decimal

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total
```

Expected result for 1,000 payments of Rs 33.40 and three adjustments of Rs 0.10: float total = 33400.30000000061; Decimal total = 33400.30.

Alternative Money Representations

Integer paise is exact when each amount can be represented in whole paise and is efficient to sum. Decimal is readable and suitable for financial calculations. Applying round() to every float addition does not eliminate the underlying binary approximation and can introduce repeated-rounding differences. Summation is O(n) time and O(1) accumulator space.

Task 3 – Variables and Constants

Python identifiers should be descriptive, begin with a letter or underscore, and contain letters, digits and underscores. Constants are conventionally written in UPPER_CASE, although Python does not enforce immutability.

```
BASE_RATE = 30.00
REFERENCE_FAT = 3.5
REFERENCE_SNF = 8.5
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STEP_SIZE = 0.1
```

Python is dynamically typed: a name can first refer to an integer (value = 10) and later to a string (value = 'ten'). Multiple assignment allows swapping with a, b = b, a.

Repeating magic numbers across k expressions means a rate revision may require k edits. Named constants reduce this to one controlled edit. A RATE_CARD dictionary groups related values and supports seasonal selection. Runtime and memory differences are negligible for a small rate card; maintainability is the key advantage.

Task 4 – Correct Expressions

```
fat_steps = round((fat - REFERENCE_FAT) / STEP_SIZE)
snf_steps = round((snf - REFERENCE_SNF) / STEP_SIZE)
rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
amount = litres * rate
```

The fat and SNF step counts are measured relative to the reference readings. The rate is calculated by adding the two adjustments to the base rate, and the delivery amount is litres multiplied by the rate.

Unexpected Expression Results

- (4.2 - 3.5) / 0.1 may produce 7.000000000000001 because decimal fractions are approximated in binary floating-point.
- 0.1 + 0.2 == 0.3 is False because the internal floating-point values differ slightly.
- -7 // 2 is -4, while int(-7 / 2) is -3: floor division rounds down, whereas int() truncates toward zero.
- round(2.5) returns 2 because Python uses ties-to-even for exact halfway cases.

For fat = 4.2, the intended step count is +7; for fat = 3.1, it is -4. int() and math.floor() can produce different results for negative non-integer values. Each expression has O(1) time and O(1) space complexity. Chained comparisons such as 2.0 <= fat <= 7.0 simplify validation.

Task 5 – Read Input and Convert Types

The input() function always returns a string. Convert appropriate values using int(), float(), or Decimal(). Invalid text such as 'eight' and '8,5' raises ValueError under the expected decimal-point format, while ' 8.5 ' is accepted by float(). Calling strip() makes whitespace handling explicit.

```
from decimal import Decimal, InvalidOperation

def read_decimal(prompt):
    while True:
        text = input(prompt).strip()
        try:
            return Decimal(text)
        except InvalidOperation:
            print("Invalid number. Please try again.")
```

A try/except ValueError or InvalidOperation loop is generally clearer and more robust than manually checking strings. Aligned output can be printed with an f-string such as f'{amount:8.2f}'. Validation per entry is O(1); processing n entries is O(n).

Task 6 – Organise Logic into Functions

```
def rate_per_litre(fat, snf):
    """Return the calculated price per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(30.00 + fat_steps * 0.40 + snf_steps * 0.30, 2)

def payout(litres, fat, snf):
    """Return the payment for one delivery."""
    return round(litres * rate_per_litre(fat, snf), 2)

def ten_day_total(payments):
    """Return the sum of recorded delivery payments."""
    return round(sum(payments), 2)
```

rate_per_litre() returns a rate, payout() returns one delivery amount, and ten_day_total() returns a period total. Returning values makes functions reusable and testable; printing only displays results. Parameters are local names. Constants may be read within functions, but changing global policy values inside functions should be avoided. Docstrings describe the function’s purpose and return value.

A long script duplicates logic and is harder to test. Lambda expressions suit short expressions, not multi-step validation. Function calls add small constant overhead, while processing n entries remains O(n).

Task 7 – Build a Module

The file milkrate.py should contain the rate card and reusable calculation functions. import milkrate uses qualified names; from milkrate import payout imports a specific name; import milkrate as mr uses an alias.

```
"""Rate card and payment functions for the milk co-operative."""
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def rate_per_litre(fat, snf):
    """Return the price per litre rounded to two decimals."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE
                 + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Return the payment for one delivery."""
    return round(litres * rate_per_litre(fat, snf), 2)

if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5),
                             (8.5, 4.2, 8.7),
                             (12, 3.1, 8.3)]:
        rate = rate_per_litre(fat, snf)
        print(f"{litres:5} L  fat {fat}  snf {snf}  "
              f"rate {rate:6.2f}  pay {payout(litres, fat, snf):8.2f}")
```

Litres	Fat %	SNF %	Rate (Rs/L)	Payment (Rs)
10	3.5	8.5	30.00	300.00
8.5	4.2	8.7	33.40	283.90
12	3.1	8.3	27.80	333.60

The __name__ == '__main__' guard runs the demonstration only when milkrate.py is executed directly. When imported, __name__ is 'milkrate', so the table is not printed. Without the guard, importing would cause unwanted output. Python searches the script/current directory and entries in sys.path. Imported modules are cached in sys.modules.

Copying functions into every script means every rate revision must be repeated. A shared module needs one update; its import has a one-time startup cost and calculations remain O(n) for n entries.

Task 8 – Algorithmic Problem Solving

Pseudocode

```
START
SET total = 0.00
FOR each delivery entry
    READ farmer_id, litres, fat, snf, shift
    IF conversion fails OR required values are invalid
        DISPLAY validation error
        CONTINUE
    END IF
    CALCULATE fat_steps and snf_steps
    CALCULATE rate from base rate and adjustments
    ROUND rate to two decimal places
```

```
    CALCULATE amount = litres * rate
    ROUND amount to two decimal places
    ADD amount to total
    DISPLAY rate and amount
END FOR
DISPLAY total
END
```

Validation Flow

Start → Read entry → Are numeric conversions successful? If no, display an error and read again. If yes, validate quantity, fat range and shift. If invalid, display an error and retry. If valid, calculate rate and amount, accumulate the rounded payment, and continue to the next entry or display the final total.

Hand Trace

Entry	Calculation	Payment
1	10 L; fat 3.5%; SNF 8.5%; rate Rs 30.00	Rs 300.00
2	8.5 L; fat 4.2%; SNF 8.7%; rate Rs 33.40	Rs 283.90
3	12 L; fat 3.1%; SNF 8.3%; rate Rs 27.80	Rs 333.60
Total	Sum of rounded payments	Rs 917.50

Storing all records and adding them later takes O(n) time and O(n) space; it is useful for audits, corrections and ten-day grouping. A running total takes O(n) time and O(1) extra space, but does not retain the underlying records.

Task 9 – Implement and Test the Calculator

The calculator should validate each entry before calculating payment. Automated assert tests should be used so that the same cases can be rerun after every change.

Test case	Expected result	Status
10 L, fat 3.5, SNF 8.5	Rs 300.00	Pass in sample calculation
8.5 L, fat 4.2, SNF 8.7	Rs 283.90	Pass in sample calculation
12 L, fat 3.1, SNF 8.3	Rs 333.60	Pass in sample calculation
Fat = 2.0, SNF = 8.5	Rate Rs 24.00	Boundary case
Fat = 7.0, SNF = 8.5	Rate Rs 44.00	Boundary case
Negative quantity	Reject entry	Validation required
Fat = 7.1	Reject entry	Validation required
Quantity = 'eight'	Reject conversion	Validation required
Empty farmer ID / invalid shift	Reject entry	Validation required

A five-entry hand total should be calculated by adding each rounded delivery payment. Manual testing requires repeated typing and is more error-prone; automated assertions are repeatable and faster after code changes. Actual execution screenshots should be added to the final submission.

Task 10 – Compare Expectations with Execution

A benchmark should compare float and Decimal summation for 300, 6,000 and 150,000 payments using time.perf_counter(). Record actual totals, differences and elapsed time on the computer used for the assignment. Timings depend on the processor, Python version and system load, so measured results should not be invented or treated as universal.

```
from decimal import Decimal
from time import perf_counter

def benchmark(n):
    values = ["33.40"] * n

    start = perf_counter()
    float_total = sum(float(x) for x in values)
    float_seconds = perf_counter() - start

    start = perf_counter()
    decimal_total = sum((Decimal(x) for x in values), Decimal("0.00"))
    decimal_seconds = perf_counter() - start

    print(n, float_total, decimal_total,
          float_seconds, decimal_seconds)

for count in (300, 6000, 150000):
    benchmark(count)
```

The accounting rule recommended here is to round each delivery payment to the nearest paisa and then sum those rounded payments. Rounding only the final aggregate can disagree when individual payments contain fractions of a paisa. Decimal is slower than float in many cases but provides decimal arithmetic when constructed from strings.

A dictionary cache for previously seen (fat, SNF) pairs can provide expected O(1) lookup, with O(k) extra space for k unique pairs. Since calculating one rate is already O(1), caching should be used only if measurement demonstrates a meaningful benefit.

Task 11 – Recommend and Defend the Final Design

Use Decimal values constructed from input strings for analyser readings and monetary calculations. Define a clear rounding policy, such as ROUND_HALF_UP, at the payment boundary. Keep policy values in one rate-card module or a validated JSON file. Each rate revision should be reviewed, versioned and tested as one change.

Limitations include the unspecified SNF range, tie-breaking rule for analyser step calculations, correction handling, database persistence, concurrent access and official seasonal calendar rules. These assumptions must be confirmed with the co-operative. At district-union scale, processing remains O(n), but a production system should include database transactions, audit logs, access controls, backups and batch processing.

If rates are read from JSON, the program becomes easier to update without editing source code, but it must validate the file, handle missing or invalid values, and record which rate-card version was used for each payment. Reading t values at startup takes O(t) time and O(t) storage.

Task 12 – Re-evaluate and Adapt the Design

A. Impact Analysis

The automatic analyser supplies fat and SNF values to two decimal places. From April to June, the base rate changes to Rs 32.00 and each fat step is worth Rs 0.45. A farmer supplying more than 500 litres over ten days receives a bonus of Rs 0.50 per litre. The rate constants, step calculations and payout logic must therefore be modified.

B. Evaluation of the Original Design

The original functions and module remain appropriate because they separate the rate calculation from payment calculation. However, the rate card must be selected by season, the bonus must be calculated from the farmer's ten-day volume, and the half-step policy must be documented.

C. Adapted Design

```
from decimal import Decimal, ROUND_HALF_UP

CENT = Decimal("0.01")
STEP = Decimal("0.1")

RATE_CARDS = {
    "regular": {"base": Decimal("30.00"),
                "fat_step": Decimal("0.40"),
                "snf_step": Decimal("0.30")},
    "apr_jun": {"base": Decimal("32.00"),
                "fat_step": Decimal("0.45"),
                "snf_step": Decimal("0.30")}
}

def whole_steps(reading, reference):
    """Round a reading difference to the nearest 0.1% step."""
    return ((reading - reference) / STEP).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf, season="regular"):
    """Return rate per litre for the selected season."""
    card = RATE_CARDS[season]
    fat_steps = whole_steps(fat, Decimal("3.5"))
    snf_steps = whole_steps(snf, Decimal("8.5"))
    rate = (card["base"] + fat_steps * card["fat_step"]
            + snf_steps * card["snf_step"])
    return rate.quantize(CENT, rounding=ROUND_HALF_UP)

def payout(litres, fat, snf, season="regular",
           ten_day_litres=Decimal("0")):
    """Return rounded delivery payment including eligible bonus."""
    base = litres * rate_per_litre(fat, snf, season)
    bonus = (litres * Decimal("0.50")
             if ten_day_litres > Decimal("500") else Decimal("0.00"))
    return (base + bonus).quantize(CENT, rounding=ROUND_HALF_UP)
```

D. Worked Tests and Defence

Scenario	Calculation	Result
Regular season, 10 L at reference readings	10 × Rs 30.00	Rs 300.00
April–June, 10 L at reference readings; 300 L in ten days	10 × Rs 32.00; no bonus	Rs 320.00

April–June, fat 4.25, SNF 8.55; 20 L; ten-day total 520 L	Rate = 32 + (8 × 0.45) + (1 × 0.30) = Rs 35.90; base Rs 718.00 + Rs 10.00 bonus	Rs 728.00
Same 20 L delivery; ten-day total exactly 500 L	No bonus because quantity must be strictly greater than 500 L	Rs 718.00

Decision: MODIFY the original design. Reusing the calculation functions avoids unnecessary redesign, while season selection and a separate bonus rule meet the new requirements. The example assumes the SNF step value remains Rs 0.30 because the changed requirement does not specify a new value. The co-operative must confirm this assumption and the ROUND_HALF_UP tie rule. Actual program output screenshots should be attached as execution evidence.

Submission Checklist

- Create repository CSA08-[RegisterNumber]-Assignment1 and add the working GitHub link to the cover page.
- Keep separate folders for Tasks 1–12, including source code, input files and output screenshots.
- Include README.md with student name, register number, Python version, libraries and run commands.
- Commit progressively, with at least one commit for each task.
- Run all tests and benchmarks on the target computer; insert actual outputs and timing results.
- Confirm the register number and update the cover page before submission.